IED-GCN landslide susceptibility

The paper, our reproduction of it, and our own variant: what each one does, how they were built, and how they compare

> **How to read this document.** Part 1 explains the whole project from zero, for someone who has never seen it. Part 2 is the paper (blue). Part 3 puts the paper next to our two models (blue = the paper, teal = ours). Part 4 lists every test we ran and what it showed. Part 5 lists what is missing, in plain terms.

The paper: Y. Li, T. Chen, L. Lv, R. Niu, A. Plaza, *IED-GCN: An Internal and External Decoupled Graph Convolutional Network for Landslide Susceptibility Assessment*, IEEE Transactions on Geoscience and Remote Sensing, vol. 63, 2025, article 4414717.

Our two models. **Paper model**: the paper's method reproduced as published. **Our model**: the same network with two changes of our own (one-hot encoded categorical factors, and a better way to cut the map into five classes). Both run on the same data with the same superpixels, splits and settings, so any difference between them comes from those two changes alone.

Part	What is in it
1	The project in plain words: rasters, pixels, superpixels, graphs, the map, and how we measure success
2	The paper: data, methods and every result table
3	Side by side: the paper, our paper model, our model; and how each was built
4	The tests we ran and what they showed, with the final scores and maps
5	What we are missing, honest conclusions, and how to describe the project

Part 1. The project in plain words

1.1 What are we trying to do?

A **landslide susceptibility map** colours every place in a region by how likely it is to have a landslide, judged from the ground itself: how steep it is, what rock it is made of, how close to a river, and so on. It answers *where* the ground is prone to sliding. It does **not** say *when* a landslide will happen, and it uses no live data. Planners use such maps to decide where to build roads and houses and where to look more closely.

The paper builds this map with a type of neural network that works on *graphs*. We reproduced it, tested it, and tried to improve it a little. This document explains all of that, starting from the data.

1.2 What goes in: rasters

A **raster** is a map stored as a grid of small squares called **pixels**, each holding one number. A photo is a raster whose pixels hold colours. In this project every pixel is a **28.5 m × 28.5 m** square of ground, and there is one raster for each **landslide conditioning factor** (a property of the ground that may influence landslides). We have **16 factors**, so every pixel carries 16 numbers, like a 16-colour photo. The model reads them as one block of size 556 × 1,885 × 16.

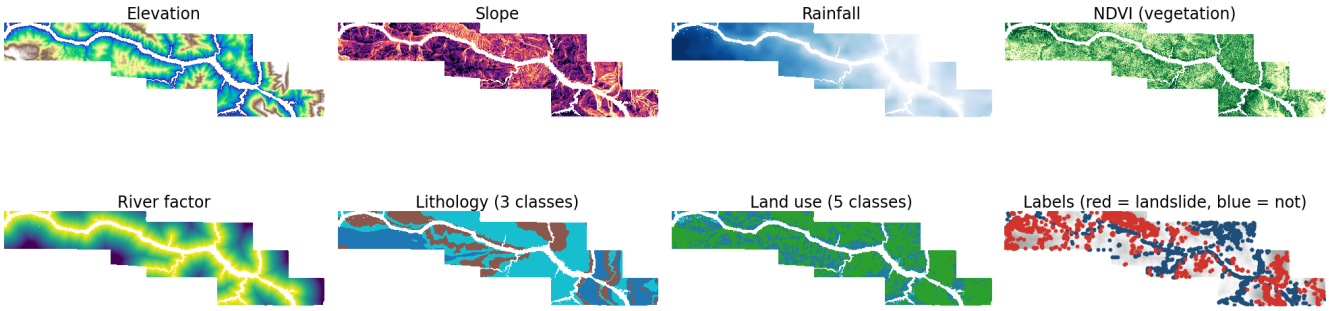


Figure 1. Seven of the 16 factor rasters and the landslide labels (last panel), drawn from our data. White areas are no-data (explained in 1.3). Colours are only for display; each factor is scaled to run from 0 to 1 before use, so no factor dominates just because its numbers are larger.

Factor (file name)	Plain meaning (read from the file name)
elevation	Height above sea level
slope	How steep the ground is
aspect	Compass direction the slope faces (8 directions)
plan, profile	Curvature across and along the slope (does the ground bulge or hollow?)
rough	How rugged the terrain is
terrain	A terrain-shape index (probably terrain position)
spi, twi	Stream power index (erosive power of flowing water); wetness index (where water collects)
rainfall	Rainfall amount
ndvi	How green and dense the vegetation is (from satellite images)
river, road, fault	Closeness to the nearest river, road and geological fault
lithology	Rock type (3 classes)
landuse	Land cover type (5 classes)

The meanings come from the file names; the dataset has no description file, so treat the exact definitions (for example distance versus density) as assumed. The paper uses 13 factors; ours has 16 (see Part 3).

1.3 How much area does it cover?

The factor rasters are each 1,885 pixels wide and 556 pixels tall, which is 1,048,060 pixels, or a 53.7 km × 15.8 km rectangle of 851 km². But the rectangle is padded: **54% of it is blank no-data** (every factor equals 0), which covers the empty corners and the surface of the Yangtze river. The real study area is **481,929 pixels = 391 km²**. The paper's study area is 396 km², also at 28.5 m, and our coordinates (about 110.31–110.88°E, 30.92–31.06°N) lie in the same Three Gorges Reservoir region, so this is almost certainly the paper's own study area, though nothing in the project says so.

Why this matters. The blank block is part of the raster the model sees. It must be left out when we count how much of the *area* falls in each susceptibility class; otherwise 'very low' looks huge. We found this late and fixed it: every area percentage in this document is measured over the real study area only. The accuracy scores were never affected, because every label lies on a real pixel.

1.4 The answer key: labels

To learn, the model needs places where we already know the answer. We have **1,523 pixels marked landslide (1)** and **1,523 pixels marked non-landslide (0)**: 3,046 labeled pixels, only 0.63% of the 481,929 real pixels. The other 99.4% are **unlabeled**: we do not know their answer. The model learns from the labeled pixels and then fills in the unlabeled ones, which is how a full map appears.

The labeled pixels are split 70/30: 70% to learn from (training) and 30% kept hidden to check the model afterwards (testing). The paper balances landslide and non-landslide samples 1:1, and so do we. **We do not know how the non-landslide pixels were chosen** (the project has no record of it), and that choice strongly affects how easy the task is.

1.5 From pixels to superpixels

A single 28.5 m pixel says little: a landslide is an area, and a pixel does not know what is around it. The paper therefore groups neighbouring pixels that look alike into small blobs called **superpixels**, using an algorithm named **SLIC**. The setting **np** is the typical number of pixels per superpixel. The paper uses **np = 10**, and so do both our models.

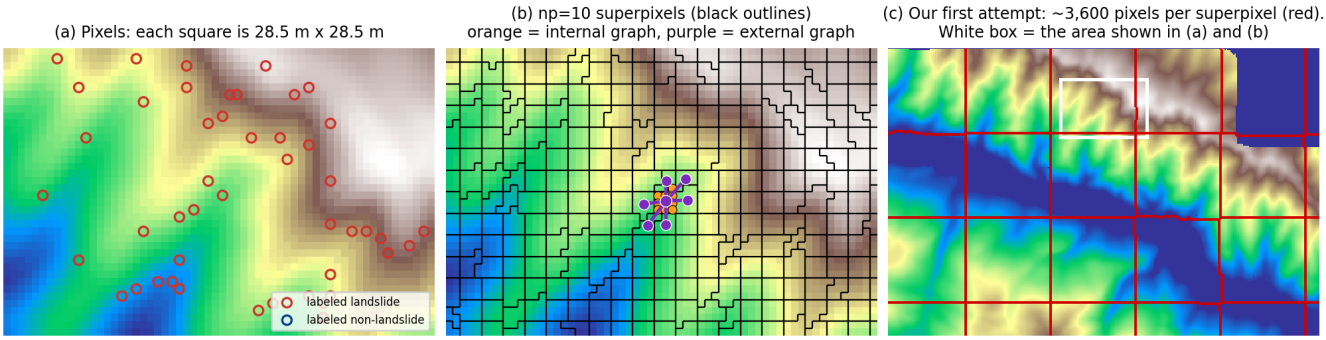


Figure 2. (a) Raw pixels around some labeled points. (b) The same area cut into np=10 superpixels (black outlines). One superpixel is highlighted: orange dots and lines are its pixels (the internal graph); purple dots are its centre and its touching neighbours (the external graph). (c) Our first attempt used about 3,600 pixels per superpixel, so one red block is larger than the whole area shown in (a) and (b).

With np=10 the rectangle becomes 115,747 superpixels, of which 53,074 lie inside the study area. Our 3,046 labeled pixels fall into **2,344 labeled superpixels** (several labeled pixels can share one superpixel); none contains both a landslide and a non-landslide pixel. A labeled superpixel takes the label of its labeled pixels.

What we are missing at this step:

- **Unlabeled pixels.** About 99.4% of the area has no label. Unlabeled superpixels are never learned from; they are only predicted.
- **The other pixels of a labeled superpixel.** A superpixel with one labeled pixel is treated as entirely that class, an approximation that is reasonable at about 10 pixels but poor at 3,600.
- **Our first attempt was far too coarse.** With about 3,600 pixels per superpixel there were only 288 superpixels, of which 161 were labeled, so the test set had 49 samples. The paper's np=10 gives 2,344 labeled superpixels.

1.6 Graphs and the network, in plain words

A **graph** is dots (*nodes*) joined by lines (*edges*). A **graph convolutional network (GCN)** lets every node update its own numbers by averaging them with its neighbours', so each node learns about its surroundings. The paper's idea is to use **two graphs at two scales** and then combine them:

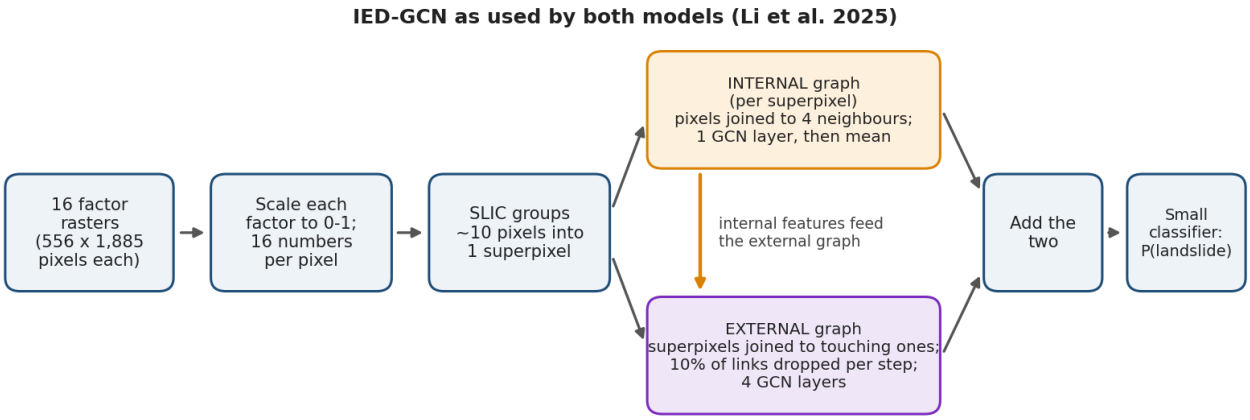


Figure 3. The IED-GCN pipeline, identical in both of our models.

- **Internal graph (small scale).** Inside each superpixel, every pixel is a node joined to its 4 neighbours (up, down, left, right). One GCN layer mixes the pixels' 16 numbers with their neighbours', then the pixels are averaged into one summary vector for the superpixel.
- **External graph (large scale).** Each superpixel becomes one node, joined to the superpixels it touches. Those summary vectors pass through 4 more GCN layers so each superpixel learns about the wider landscape.
- **Edge decoupling.** During training, a random 10% of the links in the external graph are switched off at every step. This stops the model from relying on particular links (reducing overfitting). At prediction time all links are used.
- **Combine and decide.** The internal and external summaries are added together and a small classifier turns them into the probability that the superpixel is a landslide. The network is trained on the labeled superpixels for 150 rounds (epochs).

1.7 From probabilities to a map

The model gives every superpixel a probability between 0 and 1. Painting each superpixel's probability back onto its pixels gives a probability map. For reading, the probabilities are cut into **five classes**: very low (VeL), low (Lo), moderate (Mo), high (Hi), very high (VeH). The paper cuts them with **natural breaks** (Jenks), which places the class boundaries where the values naturally cluster. Our model applies the same method to the logit of the probability (a stretched version of it); see Part 3.

1.8 How we judge the result

We hide the 30% test samples and compare the model's answers with the truth. The terms below appear throughout. All run from 0 to 1 and higher is better. Example: 100 test places, 50 with a landslide.

Score	Plain meaning
Accuracy	Share of all test places answered correctly. 0.98 means 98 of 100.
Precision	When the model says 'landslide', how often it is right. 0.97 means 3 in 100 alarms are false.
Recall	Of the real landslides, how many it found. 0.99 means it missed 1 in 100.
F1	One number combining precision and recall.
AUC	Pick one landslide place and one safe place at random: how often the model gives the landslide place the higher risk. 0.5 is a coin flip.
Kappa, MCC	Accuracy after discounting what pure guessing would give. 0 is chance, 1 is perfect.
Landslide density (LD)	For each of the five classes: the share of real landslides in it divided by the share of the area it covers. A good map has LD rising steadily from very low to very high.

Two ways to hold data out. *Random split* (what the paper does): test places are chosen at random, so a test place can sit right next to a training place, which flatters the score. *Spatial split* (our extra, stricter test): whole blocks of land about 4 km across are held out, so the model is tested on places it has never been near.

Part 2. The paper

2.1 In one paragraph

Earlier graph networks for landslide mapping build their graph from a single kind of unit, either pixels (fine detail, little context) or superpixels (context, but each treated as uniform). The paper's **IED-GCN** uses both: an **internal** graph of pixels inside each superpixel and an **external** graph of neighbouring superpixels, with random **edge dropping** on the external graph to curb overfitting. On a landslide dataset from the Three Gorges Reservoir area it reports AUC 0.9838, accuracy 0.9838, F1 0.9839 and kappa 0.9675, ahead of CNN, ResNet, DenseNet and miniGCN.

2.2 Study area and data

- **Area:** Three Gorges Reservoir area, Hubei, China (Zigui and Badong), about **396 km²**; elevation 80–2,000 m; subtropical monsoon climate, about 1,150 mm of rain a year, mostly June–September.
- **Landslides:** an inventory of **202 landslides covering 23.40 km²**, built from satellite image interpretation, field surveys and historical records, concentrated along the Yangtze reservoir banks.
- **Pixel size:** everything resampled to **28.5 m × 28.5 m**; all factors normalised to a common numeric range.
- **13 factors:** elevation, slope, aspect, bedrock slope, TPI, TRI, SPI, TWI (all from an ALOS 30 m terrain model); lithology and distance to fault (geological map); distance to road and distance to river (OpenStreetMap); NDVI (Landsat-8).

2.3 Checking the factors

The authors tested whether any factors overlap too much (**VIF**; a value above 10 means a problem) and ranked their usefulness (**Gini importance**). All 13 had VIF below 10 (largest: slope 7.89, TPI 4.56), so all were kept. The most useful were **lithology, bedrock slope and elevation**; the least useful were SPI and TWI.

2.4 Building the training data

- Landslide and non-landslide samples in equal numbers (1:1), mixed into one dataset.
- Random **70/30** split into training and test sets.
- Data augmentation on the training set: added noise and 'colour jittering'.

2.5 The model, step by step

- **Superpixels:** SLIC cuts the study area into S = (height × width) / np superpixels, with **np = 10** pixels each.
- **Internal graph convolution (IGC):** pixels inside a superpixel are nodes joined to their 4 neighbours. Their factor values are mixed with the neighbours' (Eq. 6), then averaged (mean pooling, Eq. 5) into one vector per superpixel. One layer.
- **External graph convolution (EGC):** superpixels are nodes, joined if they touch. Four layers of 'ReLU(normalised neighbours × features × weights)' (Eq. 10). More layers would blur everything together (over-smoothing).
- **Edge decoupling:** in each training step every external link is kept with probability 0.9 and dropped with probability 0.1 (Eqs. 11–12).
- **Fusion and output:** H_all = H_in + H_ex (Eq. 13), then a small multilayer network gives the landslide probability (Eq. 14).
- **Training safeguards:** cross-entropy loss plus an L1 penalty on the weights, and gradient clipping.

Setting	Paper value
Hidden units	128
Layers	1 internal GCN layer, 4 external GCN layers
Epochs	150
Optimizer	Adam, learning rate 0.001, weight decay 0.001
Edge discard rate	0.1
Batch size	64
Superpixel size np	10 (also tried 70 and 145)
Machine	RTX 3090 GPU, Ubuntu 18.04, Python 3.9, PyTorch 1.10.1

2.6 The paper's results

Main comparison (Table VI; random 70/30 split; best in bold in the paper).

Model	Accuracy	Precision	Recall	F1	MCC	Kappa	AUC
CNN	.9507	.9209	.9861	.9524	.9037	.9014	.9442
ResNet	.9654	.9511	.9811	.9659	.9312	.9307	.9627
DenseNet	.9595	.9395	.9823	.9604	.9200	.9190	.9588
miniGCN	.9720	.9584	.9869	.9724	.9445	.9440	.9753
IED-GCN	**.9838**	**.9772**	**.9907**	**.9839**	**.9676**	**.9675**	**.9838**

Ablation (Table VII): what each part adds.

Variant	IGC	EGC	Edge drop	Accuracy	F1	Kappa	AUC
Baseline GCN				.9634	.9640	.9268	.9640
IGC only	yes			.8651	.8699	.7303	.8654
EGC only		yes		.8868	.8934	.7736	.8868
IE-GCN (no edge drop)	yes	yes		.9810	.9812	.9621	.9810
IED-GCN	yes	yes	yes	**.9838**	**.9839**	**.9675**	**.9838**

Each module alone is worse than a plain GCN; together they beat it; edge dropping adds the last step.

Edge discard rate (Table VIII), **superpixel size** (Table IX), **class balance** (Table X):

Edge discard rate	Accuracy	F1	Kappa	AUC
0%	.9810	.9812	.9621	.9810
10%	**.9838**	**.9839**	**.9675**	**.9838**
20%	.9807	.9809	.9615	.9806
30%	.9801	.9802	.9602	.9802

Superpixel size np	Accuracy	F1	Kappa	AUC
10	**.9838**	**.9839**	**.9675**	**.9838**
70	.9353	.9375	.8705	.9353
145	.8818	.8889	.7635	.8818

Landslide : non-landslide	Accuracy	Recall	Kappa	AUC
1:1	.9838	**.9907**	**.9675**	**.9838**
1:2	.9797	.9773	.9545	.9791
1:4	.9806	.9598	.9397	.9728
1:8	.9838	.9223	.9177	.9569
1:16	.9879	.8830	.8902	.9387

Smaller superpixels are far better (np=10 vs 145: kappa 0.9675 vs 0.7635). The more the landslide samples are outnumbered, the more landslides are missed (recall falls from 0.99 to 0.88), even though accuracy stays high.

The paper's map (Table V, IED-GCN row): High and Very High together cover 10.25% of the area and hold 97.81% of the historical landslides, and density rises steadily with class.

Class	Share of area	Share of landslides	Landslide density
Very low	82.8%	0.1%	0.001
Low	4.0%	0.4%	0.092
Moderate	3.0%	1.7%	0.577
High	2.6%	6.8%	2.597
Very high	7.6%	91.0%	11.911

Stability. Five runs with different random non-landslide samples gave the highest AUC for IED-GCN every time, with less variation than CNN and ResNet (agreement between runs $R^2 = 0.9545$).

2.7 Limitations the authors state

- Results depend on the quality and completeness of the landslide inventory; remote areas often have incomplete ones.
- The fixed SLIC settings (size, compactness) are chosen by hand and can lose detail in rough terrain; adaptive superpixels are suggested.
- Data of different resolutions are resampled to one grid, which can add noise; adding radar or drone data is suggested.
- Not stated by the authors but visible in the paper: the test set is a random 30% of the same area, with no held-out region.

Part 3. Side by side: the paper and our two models

3.1 The three versions at a glance

Paper = what the authors published. **Paper model** = our reproduction of it. **Our model** = the reproduction plus two changes. Where Paper model and Our model differ, the cell for Our model is shaded.

	Paper (published)	Paper model (our reproduction)	Our model
Goal	Landslide susceptibility map	Same	Same
Study area	Three Gorges Reservoir, 396 km²	Same region, 391 km² of real data in a padded rectangle	Same data
Pixel size	28.5 m	28.5 m	Same
Factors	13: elevation, slope, aspect, bedrock slope, TPI, TRI, SPI, TWI, lithology, dist. to fault, road, river, NDVI	16: the 13 minus bedrock slope, plus plan and profile curvature, land use, rainfall (names differ slightly)	Same 16, but aspect, land use and lithology are one-hot encoded (29 input channels)
Labels	202 landslides; equal number of non-landslide samples	1,523 landslide + 1,523 non-landslide pixels (how chosen: unknown)	Same
Scaling	Normalised to one range	Each factor scaled 0 to 1	Same
Factor checks	VIF and Gini importance	Not done	Not done
Data split	Random 70/30	Random 70/30 of labeled superpixels; also a spatial split as an extra	Same two splits
Augmentation	Noise and colour jitter on training data	None	None
Superpixels	SLIC, np = 10	SLIC, np = 10 (115,747 in the rectangle, 53,074 in the study area)	Same superpixels
Internal graph	4-neighbour pixel graph, 1 GCN layer, mean pool	Same (propagation done once up front, which gives the same result for 1 layer)	Same
External graph	Touching superpixels, 4 GCN layers	Same	Same
Edge dropping	10% of links per step	10% of links per step	Same
Fusion	H_in + H_ex, then MLP	Same; the MLP also has 30% dropout (our addition)	Same
Loss and safeguards	Cross-entropy + L1 + gradient clipping	Same (L1 weight 1e-5, clipping at norm 2.0; the paper gives no values)	Same

	Paper (published)	Paper model (our reproduction)	Our model
Training	Adam 1e-3, weight decay 1e-3, 150 epochs, batch 64, GPU	Same, but the whole graph per step (no batches), CPU	Same
Five map classes	Natural breaks on the probability	Natural breaks on the probability, fitted on the study area only	Natural breaks on the logit of the probability, fitted on the study area only
Compared against	CNN, ResNet, DenseNet, miniGCN	Logistic regression, random forest (reference only)	Same references
Extra tests	Ablation, edge rate, np, class balance, stability	Ablation; random vs spatial split; 5 seeds; landslide density	Same, plus effect of the changes

3.2 How each model was built

Both models live in one folder, **ied_gcn_project**, and share all code except two small files:

File	What it does
data/	The inputs: features.npy (556 × 1,885 × 16 factors) and labels.npy (1 landslide, 0 non-landslide, -1 unlabeled)
common.py	Everything shared: scaling, SLIC superpixels, the internal and external graphs, the network, training, scores, natural breaks, landslide density, map pictures
paper_model.py	**Model 1.** The paper as published: raw 16 factors in, natural breaks on the probability
ours_model.py	**Model 2.** One-hot encodes aspect, land use and lithology; natural breaks on the logit
run_paper.py / run_ours.py	Click Run to train and test that model on 5 seeds × 2 splits; results go to results/paper or results/ours
run_compare.py	Click Run after both: builds results/comparison.md

What the two changes in our model do, and why.

- **One-hot encoding.** Lithology has 3 classes, land use 5 and aspect 8. In the paper model they are numbers 0–1, which pretends that 'class 3' is three times 'class 1'. One-hot encoding gives each class its own on/off column, so the network treats them as labels. It changes only the network input: the superpixels are still cut from the 16 original factors, so both models see identical superpixels.
- **Natural breaks on the logit.** The trained network pushes almost every probability to nearly 0 or nearly 1. Natural breaks on those raw values then leaves the middle classes almost empty. The logit stretches the ends out, so the breaks fall in more useful places. This changes the map only, not the predictions or the scores.

Part 4. The tests we ran and what they showed

4.1 The list of tests

#	Question	What we did	Result
1	How good was our first attempt?	Old code: 288 superpixels (about 3,600 pixels each), random split, 150 epochs	Accuracy 0.878, recall 0.833, kappa 0.752, but tested on only 49 superpixels, so unreliable
2	Do simple models already solve this data?	Logistic regression and random forest on the 16 raw factors, 5 seeds, both splits	Yes, almost: logistic regression AUC 0.998, random forest AUC 1.000 and accuracy 0.996 (random split). The labels are easy
3	Does superpixel size matter?	Early exploratory single runs, spatial split (not kept as files)	About 3,800 px: AUC 0.853, kappa 0.50. About 65 px: AUC 0.995, kappa 0.95. np=10: AUC 0.9998, kappa 0.994. Same direction as the paper's Table IX
4	Do both graphs matter?	Earlier ablation, 3 seeds, spatial split (files not kept)	Internal graph only: kappa 0.917. Internal + external: 0.975. With edge dropping: 0.969 (5 seeds). The external graph clearly helps
5	Does edge dropping help?	Same ablation	No visible gain on our data (0.975 without vs 0.969 with, within the run-to-run spread). The paper reports a small gain
6	Final scores of both models	5 seeds × random and spatial splits each, 150 epochs	See 4.2
7	Does averaging 3 networks help?	Tried an ensemble inside our model	No: spatial kappa 0.9691 (paper model), 0.9783 (one-hot only), 0.9776 (one-hot + ensemble). Dropped, since it costs three times the time
8	Do the map classes behave?	Landslide density by class on held-out landslides, natural breaks on probability vs logit	See 4.3
9	Is all the raster real data?	Counted pixels where every factor is at its minimum	No: 54% is blank no-data. Area statistics and class breaks were corrected and both models re-run
10	Do the click-to-run scripts work?	Ran all three from the system Python in a throw-away copy with a quick 2-epoch setting	All three ran to the end and wrote every output file

4.2 Final scores

Mean over 5 seeds, with the spread (± standard deviation) underneath. Each seed uses a different random draw of the split. The first row is the paper's published result on its own data.

Model	Split	AUC	Accuracy	Precision	Recall	F1	Kappa
Paper, as published (its own data)	random	0.9838	0.9838	0.9772	0.9907	0.9839	0.9675
Paper model	random	0.9998 ± 0.0001	0.9949 ± 0.0011	0.9952 ± 0.0015	0.9961 ± 0.0029	0.9957 ± 0.0010	0.9894 ± 0.0023
Our model	random	0.9999 ± 0.0000	0.9949 ± 0.0021	0.9957 ± 0.0017	0.9956 ± 0.0029	0.9957 ± 0.0018	0.9894 ± 0.0044
Paper model	spatial	0.9991 ± 0.0009	0.9857 ± 0.0141	0.9987 ± 0.0011	0.9755 ± 0.0251	0.9868 ± 0.0132	0.9691 ± 0.0277
Our model	spatial	0.9997 ± 0.0002	0.9900 ± 0.0054	0.9978 ± 0.0015	0.9844 ± 0.0116	0.9910 ± 0.0056	0.9777 ± 0.0110

For reference, logistic regression scores AUC 0.998 / accuracy 0.980 / kappa 0.961 (random split) and 0.995 / 0.965 / 0.924 (spatial); random forest 1.000 / 0.996 / 0.992 (random) and 0.999 / 0.971 / 0.941 (spatial).

4.3 The maps and landslide density

White areas are no-data. Black dots are the labeled landslide pixels. Both maps come from the seed-0 random-split run.

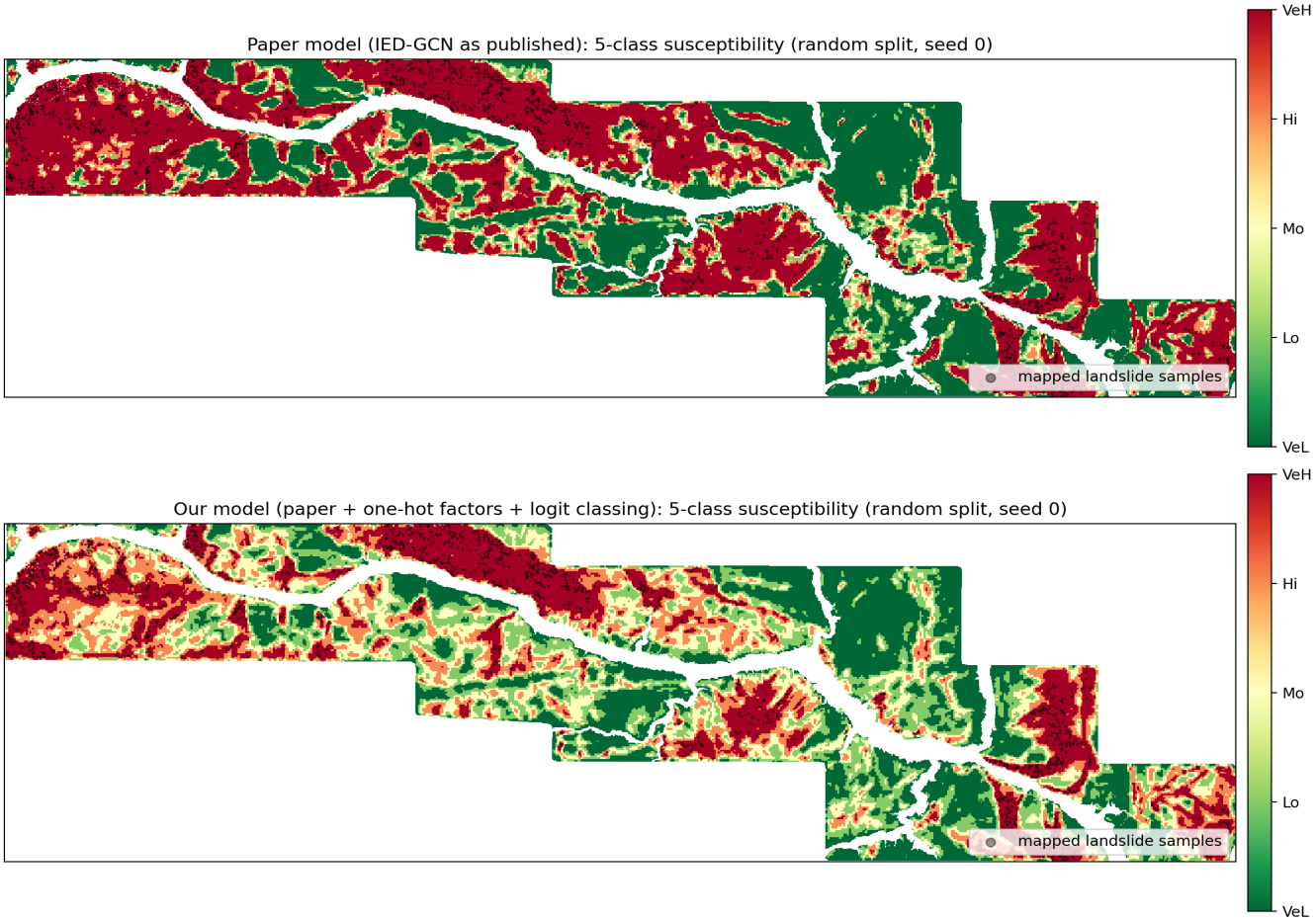


Figure 4. Top: paper model. Bottom: our model.

Random split (held-out landslides, seed 0). Area is the share of the real study area in each class; density is (share of landslides) / (share of area). In the last row the two numbers are the area share, then the share of landslides held in High + Very high. The paper's columns are its own Table V.

Class	Paper (published)		Paper model		Our model	
	area	density	area	density	area	density
Very low	82.8%	0.00	35.0%	0.00	24.3%	0.00
Low	4.0%	0.09	8.2%	0.00	18.6%	0.01
Moderate	3.0%	0.58	6.4%	0.07	18.7%	0.05
High	2.6%	2.60	8.2%	0.05	17.1%	0.61
Very high	7.6%	11.91	42.2%	2.35	21.3%	4.15
High + Very high: area / landslides	10.25%	97.81%	50.4%	99.6%	38.4%	98.9%

Spatial split (landslides in held-out blocks, seed 0; the paper has no such test).

Class	Paper model		Our model	
	area	density	area	density
Very low	34.9%	0.07	25.2%	0.02
Low	8.1%	0.32	20.6%	0.10
Moderate	6.5%	0.51	19.8%	0.29
High	8.2%	0.44	16.4%	1.91
Very high	42.3%	2.08	18.1%	3.34
High + Very high: area / landslides	50.5%	91.7%	34.5%	91.7%

4.4 What the results say

- **The reproduction reaches the paper's level on the scores.** Both our models score above the paper's published numbers, and the trends match (small superpixels win by a wide margin; internal plus external graphs beat the internal graph alone).
- **The scores are not a fair 'we beat the paper'.** The data is different and easy: plain logistic regression already reaches AUC 0.998, and a random forest matches the network on the random split.
- **Our model is not clearly more accurate than the paper model.** The score differences between them are small compared with the run-to-run spread (the ± values), so they are hints, not proofs.
- **The map is where our model differs.** On held-out landslides (random split) our model's density rises steadily across the five classes (0.00, 0.01, 0.05, 0.61, 4.15), while the paper model's high classes are almost flat before the last one (0.00, 0.00, 0.07, 0.05, 2.35). Our High + Very high class covers 38.4% of the study area, the paper model's 50.4%.
- **Both maps still flag far too much area as high risk.** The paper's High + Very high covers 10.25% of its area and holds 97.81% of the landslides; ours cover 38% (our model) and 50% (paper model) and hold 98.9% and 99.6% of the held-out landslides. A likely reason (not verified) is that our non-landslide training points are easy to tell apart, so the network learns a broad 'high, steep ground means landslide' rule and applies it to all similar ground. This is the main weakness of the project.

Part 5. What we are missing, and how to describe the project

5.1 What the paper has that we do not

- **Bedrock slope** (a top-3 factor in the paper) is not in our data; we have 16 factors instead of 13.
- **VIF and Gini factor analysis**: not done.
- **Data augmentation** (noise, colour jitter): not done.
- **Baselines CNN, ResNet, DenseNet, miniGCN**: not built; we compared with logistic regression and random forest instead.
- **Edge-discard sweep (0–30%), np sweep, class-balance sweep (1:1 to 1:16)**: not run properly. Only rough one-off checks of np.
- **Stability test with redrawn non-landslide samples**: we change the random seed but cannot redraw the non-landslide points, because we do not have the code that produced them.
- **Mini-batches of 64**: we train on the whole graph at each step.
- **Violin plots and the agreement-between-runs analysis** ($R^2 = 0.9545$ in the paper): not done.

5.2 Weaknesses of our own work

- **The data is easy.** We do not know how the non-landslide pixels were chosen; if they were drawn at random they are easy to tell apart from landslide sites. High scores partly reflect that, not only the model.
- **The maps are not as selective as the paper's.** Our High + Very high class covers 38% of the study area (paper model 50%) against the paper's 10.25%, so the density of landslides in the top class is much lower than the paper's 11.9.
- **Edge dropping showed no benefit** on our data, unlike in the paper; the data may be too easy to show it.

- **One study area.** The model learns this area's patterns; for another region it must be retrained, and it needs its own landslide inventory.
- **Some test results are from earlier one-off runs** (tests 3 and 4 in Part 4); their files were not kept, so they are indications only.
- **No live data.** This is a static map of where ground is prone to slide, not a warning system.

5.3 How to describe the project honestly

> “I reimplemented the IED-GCN paper (IEEE TGRS 2025) in PyTorch: SLIC superpixels, an internal pixel graph and an external superpixel graph, edge dropping and fusion. I trained it on a landslide dataset of the Three Gorges area with 16 conditioning factors and produced a five-class susceptibility map. My first version used superpixels that were far too large; matching the paper's setting of about 10 pixels per superpixel raised the test kappa from about 0.75 to about 0.99. I also checked it against simple baselines and with a stricter spatial hold-out test, and found the data is easy: a random forest scores about the same. I then built a variant with one-hot encoded categorical factors and a better map classing, and I report where it helps (the map) and where it does not (the scores).”

Things not to claim: that we beat the paper's accuracy (different, easier data); that this predicts when landslides happen; that edge decoupling is proven to help (we did not see it); that the project uses live data.

5.4 Where everything is

Project folder: **ied_gcn_project** (code, data, results). Results: **results/paper** and **results/ours** (scores, maps, per-run files) and **results/comparison.md**. Explanation and how to run: **README.md**. The paper: **ADD_PROJECT.pdf**. Raw factor rasters: **_extracted_dataset**.